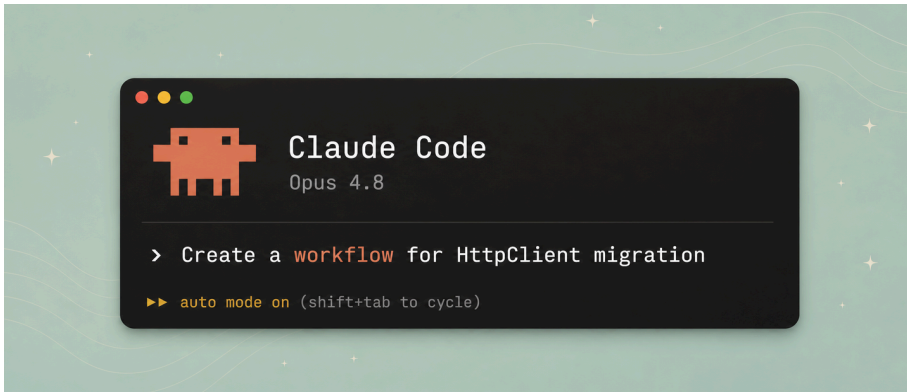




Claude Code Dynamic Workflows：把编排逻辑搬进代码的新原语



riba2534 
字节跳动 服务端研发工程师

16 人赞同了该文章 >

当任务大到一次对话装不下时，让 Claude 把编排过程写成一段脚本

5 月 28 日，Anthropic 发布了 Claude Opus 4.8，随之而来还带来了一个新功能——Dynamic Workflows（动态工作流）。

先看一个数字：11 天、约 75 万行 Rust 代码、99.8% 的原有测试通过。这是 Bun 作者 Jarred Sumner 把整个 Bun 运行时从 Zig 迁移到 Rust 交出的成绩单，而扛起这场迁移的主力，正是这个 Dynamic Workflows。

它作为研究预览（research preview）放出，瞄准的是那一类单个 Agent 一次跑不完的活，官方博客的原话是这样描述的：

Some problems are too big for one pass by a single agent, especially in complex, legacy codebases: a bug hunt across an entire service, a migration that touches hundreds of files, a plan you want stress-tested from every angle before you commit to it.

整个服务范围的 bug 排查、动辄上百个文件的迁移、一个需要从各个角度反复推敲才敢拍板的方案——这些任务的共同点是规模超出了一轮对话能协调的范围。Dynamic Workflows 给出的答案，是让 Claude 把这套编排过程写成一段可执行的脚本。

从 Subagent 到 Workflow

要理解 Workflow 的位置，得先把 Claude Code 已有的几层协作能力捋一遍。

最底层是单个 session，一个 Agent 实例从头干到尾，串行处理。往上一层是 subagent——主 Agent 派生出若干小弟去搜文件、读代码、跑命令，干完把结果汇报回来。再往上是今年早些时候推出的 Agent Teams，多个独立的 Claude Code 实例像团队一样并行协作，队员之间还能互相通信。

这几层有一个共同的瓶颈：编排者始终是 Claude 本身。它逐轮决策下一步派谁去干什么，而每一个 subagent 的返回结果，都要先回到 Claude 的上下文窗口里，它读完才能决定接下来怎么走。这套机制在任务规模不大时很灵活，可一旦要协调几十上百个并行任务，问题就来了：上下文窗口装不下那么多中间结果，Claude 的注意力也会被海量的过程信息稀释。

Workflow 换了个思路。这一次 Claude 不再亲自逐轮调度，它先把整个编排过程写成一段 JavaScript 脚本——循环、分支、中间结果的收集全都固化在脚本里，然后按顺序一行一行去执行。官方文档把这个转变概括得很精炼：

▲ 赞同 16 ▼

1 条评论

★ 43

♥ 2

➔ 分享

📄 申请转载

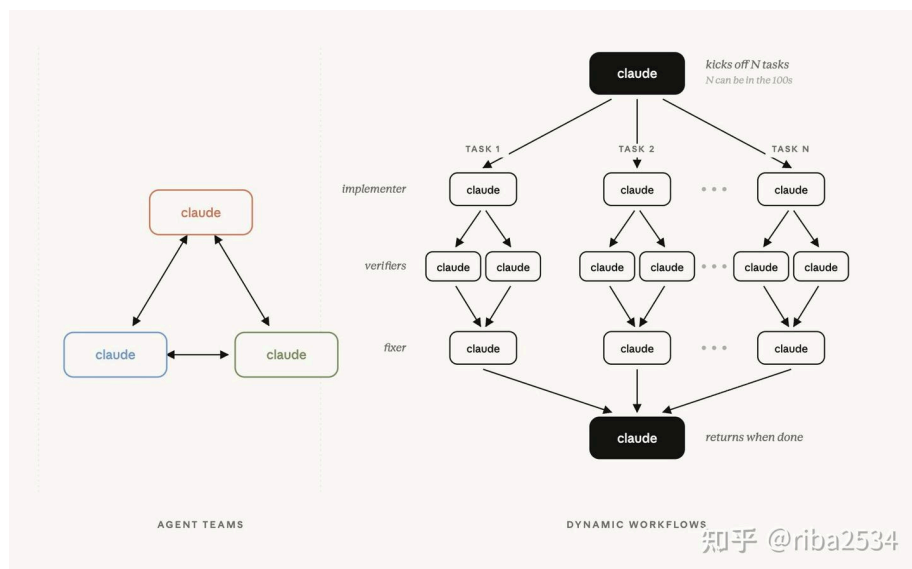




A workflow moves the plan into code. With subagents and skills, Claude is the orchestrator: it decides turn by turn what to spawn next, and every result lands in Claude's context. A workflow script holds the loop, the branching, and the intermediate results itself, so Claude's context holds only the final answer.

计划被搬进了代码。脚本自己持有循环、分支和中间结果，Claude 的上下文里只剩下最后那个答案。这跟“把大型代码库塞进有限上下文窗口”的思路是同一条路线上的延续——前者解决的是“上下文怎么省着用”，Workflow 解决的是“当工作量大到上下文根本装不下时怎么办”。

把 Agent Teams 和 Dynamic Workflows 摆在一起看：



图片出自 Anthropic 产品经理 Cat Wu 发布 Dynamic Workflows 的推文。左边 Agent Teams 是网状协作，右边 Dynamic Workflows 则是“一个 Claude 扇出上百个 task，每个 task 走 implementer → 两个 verifier → fixer 三层，最后扇入返回”的树状结构——后面 Bun 案例里“每个文件配两个 reviewer”的设计，在这张图上已经能看到雏形。

Workflow 到底在哪运行

开始拆架构之前，得先破除一个最常见的误解：很多人以为 Workflow 是某个跑在 Anthropic 服务端的编排引擎，于是去找它的 API 协议、担心第三方中转兼容性。

实际情况是：Workflow 工具本身不请求任何服务端。它是 Claude Code 在你本机跑的一段 JavaScript 编排脚本——agent()、parallel()、pipeline() 这些都是在你电脑上执行的控制流。真正去“请求服务端”的，是脚本里 agent() 调用 spawn 出来的每个 subagent，而 subagent 调用模型的方式，和你主对话窗口完全一样。

这件事有个直接推论：如果你用第三方 API 中转，Workflow 跑挂了，那跟 Workflow 没关系——它用的就是 Claude Code 平时调模型一直在用的那套接口，即 Anthropic 原生的 Messages API（不是 OpenAI 的 /v1/chat/completions）：

```
POST {ANTHROPIC_BASE_URL}/v1/messages
```

Headers:

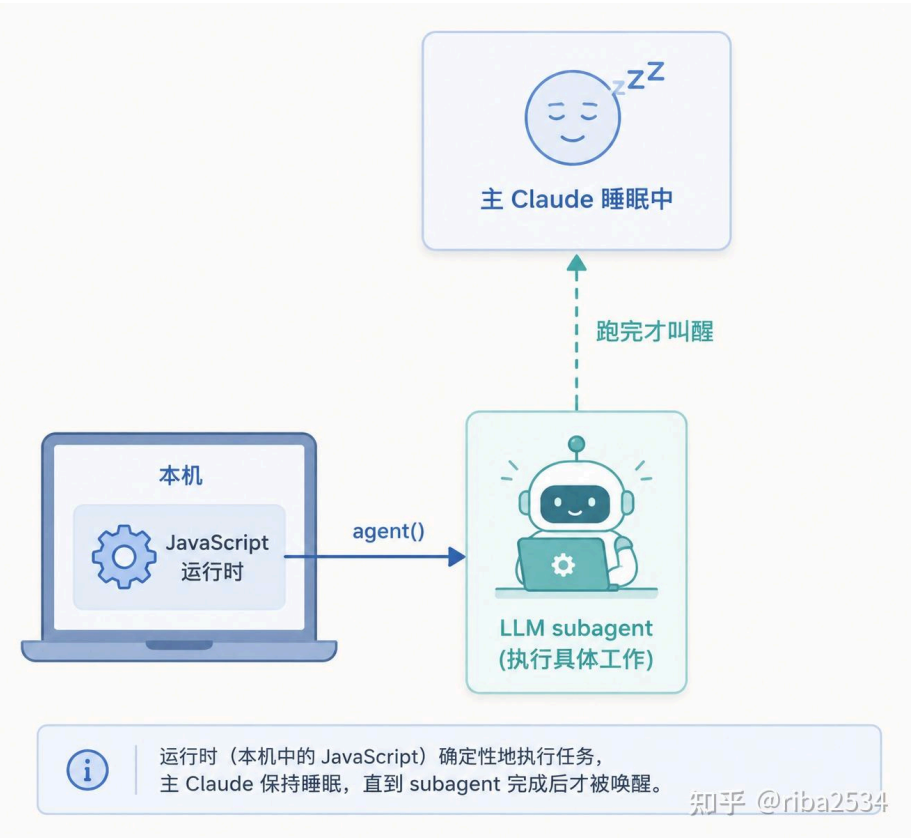
```
x-api-key: <key> # 或第三方常用的 Authorization: Bearer <token>
anthropic-version: 2023-06-01
anthropic-beta: context-1m-2025-08-07 # 1M context 会带这个 beta flag
content-type: application/json
```

Body:

```
{
  "model": "claude-opus-4-8", # 裸 model id
```

```
"system": [ {... cache_control ...} ], # system 是数组, 带 prompt cache
"messages": [...],
"tools": [...], # Claude Code 会塞几十个 tool 定义
"stream": true # 默认流式
}
```

把这层关系理清之后，Workflow 的运行模型就清晰了：一个无脑的、确定性的 JavaScript 运行时当指挥，它只会循环、拼字符串、await，本身不含任何 LLM；只有当脚本执行到 agent(...) 那一行，运行时才去临时雇一个 LLM subagent 干活。而“真正的 Agent”——也就是你正在对话的主 Claude——在脚本执行期间根本没在运行：它在发出 Workflow 调用后那一回合就结束了，脚本在后台独立跑，跑完用一条通知把它叫醒，让它去读最后的结果。

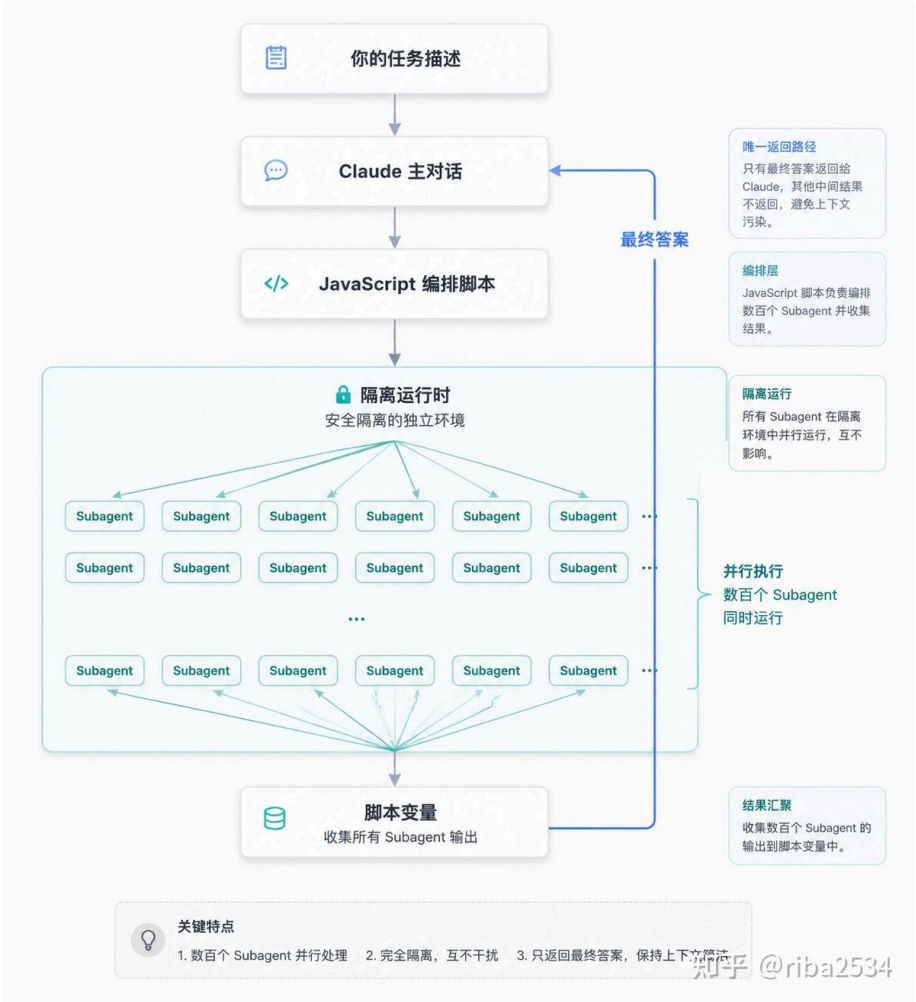


一句话概括这个分工：JavaScript 运行时当指挥（无脑、确定性），在 agent() 点临时雇 LLM 干活，主 Agent 全程在睡觉，只在最后被叫醒读结果。记住这条，后面所有的特性都好理解了。

Workflow 的核心架构

一个 Workflow 跑起来，背后是四个部件在配合：

部件	职责
编排脚本（Orchestration Script）	Claude 根据你的任务自动生成的 JavaScript，定义了整个流程的控制逻辑
隔离运行时（Isolated Runtime）	在独立于对话的环境里执行脚本，会话期间你的终端依然可以正常交互
Subagents 池	脚本调度出来的工作者，负责实际的读、写、跑命令
脚本变量（Script Variables）	中间结果的存放处，留在脚本里，不进 Claude 的上下文窗口



这张图里最关键的一条线，是 subagent 的结果先流进脚本变量，在运行时内部完成循环和校验，最后只有汇总过的答案才回到 Claude。这跟 subagent 那种“每个结果都要经过 Claude 大脑”的模式有本质区别。

官方文档给了一张三者对照表，把 Workflow 和 Subagents、Skills 的定位差异讲得很清楚：

维度	Subagents	Skills	Workflows
本质	Claude 派生的工作者	Claude 遵循的指令	运行时执行的脚本
谁在做决策	Claude 逐轮决定	Claude 按提示词决定	脚本控制流程
中间结果落在哪	Claude 上下文	Claude 上下文	脚本变量
可复用的是什么	工作者定义	指令本身	整套编排逻辑
并行规模	每轮几个	每轮几个	单次数十到数百个
中断后	重启当前轮	重启当前轮	同会话内可恢复

读这张表的时候，注意“可复用的是什么”这一行。subagent 和 skill 复用的是“一个工作者”或“一条指令”，而 Workflow 复用的是整套编排逻辑——这意味着一个调度数百个 agent 做交叉验证的复杂流程，写好一次就能存下来反复运行。这是 Workflow 区别于其它原语的地方。

脚本长什么样：meta、原语和那个最容易踩的坑

既然 Workflow 是一段 JS 脚本，那它到底长什么样？这一节把骨架拆开看。

每个脚本必须以 `export const meta = {...}` 开头，而且 meta 必须是纯字面量——不能有变量、函数调用、模板插值。它定义了脚本的名字、一行描述（会显示在权限弹窗上）和阶段划分：

```
export const meta = {
  name: 'find-flaky-tests',
  description: 'Find flaky tests and propose fixes', // 一行，显示在权限弹窗
  phases: [ // 每个 phase() 调用
    { title: 'Scan', detail: 'grep test logs for retries' },
    { title: 'Fix', detail: 'one agent per flaky test' },
  ],
}
// 脚本体从这里开始
phase('Scan')
const flaky = await agent('grep CI logs for retry markers', { schema: FLAKY })
phase('Fix')
// ...
```

meta 之后就是脚本体。能用的核心原语不多，一张表就能记住：

原语	作用	关键点
agent(prompt, opts?)	spawn 一个 subagent	不带 schema 返回字符串；带 schema (JSON Schema) 返回已校验的对象，模型不匹配会自动重试
pipeline(items, s1, s2, ...)	每个 item 独立走完所有 stage	默认选择，无屏障；item A 可以在 stage3，而 B 还在 stage1
parallel(thunks)	并发跑一批任务	屏障，等全部完成；传的是函数数组 () => agent(...), 不是 promise
phase(title)	进度分组	后续 agent 归到这一组
log(msg)	给用户发一行进度	显示在进度树上方
workflow(name, args)	内联调一个子 workflow	只能嵌一层
args / budget	外部参数 / token 预算	budget.total、budget.remaining() 可用于按预算扩展深度

每个 agent() 的提示词，就写在脚本里，是普通的 JS 字符串。要给不同 agent 喂不同的输入，常见写法是把 prompt 写成一个“返回字符串的函数”，调用时用 .map() 把循环变量插进去。数据就是这样在 agent 之间流动的——靠的就是普通的字符串拼接，没有什么消息总线：逐项数据用 .map() 插进每个 agent 的 prompt，跨阶段数据则把上一阶段的返回值 JSON.stringify 后拼进下一阶段的 prompt。

整段脚本里最容易踩的坑，是 pipeline 和 parallel 分不清。两者的本质分界是**有没有屏障 (barrier)**：parallel 会等这一批全部跑完才往下走，pipeline 则让每个 item 各自独立地流过所有 stage，互不等待。下面这种写法就是典型的浪费：

```
const a = await parallel(...) // ❌ 屏障：等全部跑完
const b = transform(a)        // 只是 flatten / map / filter，没有跨 item 屏障
const c = await parallel(b.map(...))
```

如果 5 个任务快慢不一，中间这个屏障会让快的干等慢的。正确的做法是把中间的 transform 塞进 pipeline 的一个 stage，让每条数据一旦就绪就立刻流向下一步：

```
// ✅ 维度 'bugs' 的发现可以在 'perf' 还在 review 时就开始 verify
const results = await pipeline(
  DIMENSIONS,
  d => agent(d.prompt, { label: `review:${d.key}`, phase: 'Review', schema: REVIEW }),
  review => parallel(review.findings.map(f =>
    agent(`对抗性验证: ${f.title}`, { phase: 'Verify' })
  ))
)
```

```

    .then(v => ({ ...f, verdict: v })))
  ))
)

```



只有三种情况才真正需要屏障：下一阶段前要对全集去重或合并；要根据总数提前退出（比如“0 个 bug 就跳过整个验证阶段”）；下阶段的 prompt 要引用“其他所有发现”做横向比较。除此之外，有疑问就用 pipeline。

三种触发方式

让 Claude 起一个 Workflow，有三条路子。

第一条是在 prompt 里直接出现 workflow 这个词。Claude Code 会把它高亮出来，提示你这句话可能触发一个工作流：

```
Run a workflow to audit every API endpoint under src/routes/ for missing au
```

如果你只是顺口提了一句 workflow、并不想真的触发，按 alt+w 就能忽略这个高亮。

第二条是开启 ultracode 模式，让 Claude 自己判断要不要起 Workflow：

```
/effort ultracode
```

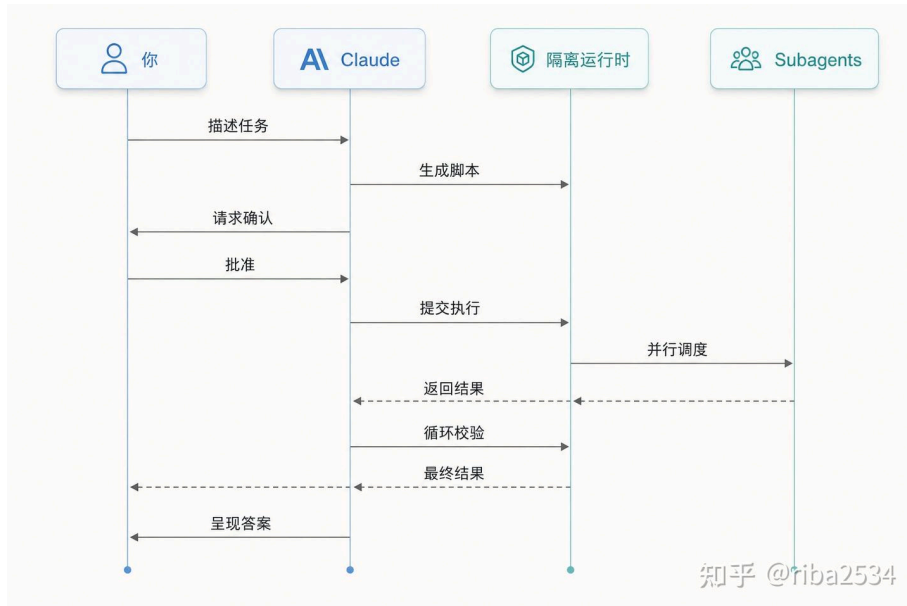
第三条是运行一个已经保存好的 Workflow，它会以斜杠命令的形式出现：

```
/deep-research What changed in the Node.js permission model between v20 and
```


保存下来的 Workflow 放在两个位置，决定了它的可见范围：

```
.claude/workflows/      # 项目级，克隆仓库的人都能用，跟团队共享
~/claude/workflows/     # 个人级，只有你能看到，但在每个项目里都能用
```

保存时按 Tab 在这两个位置之间切换。如果项目级和个人级有同名 Workflow，项目级优先。存好之后它就成了一个 `/<name>` 命令，出现在斜杠自动补全菜单里，跟普通的 slash command 混在一起。



脚本生成之后、真正跑起来之前，你有机会审一眼它要干什么。运行中按 `Ctrl+G` 还能在编辑器里打开脚本，看 Claude 到底写了段什么样的代码。这种“代码可见、可审”的特性，是 Workflow 相比黑盒式自动化更让人放心的地方。

执行模型与那些硬约束

Workflow 的运行时跟你的对话是隔离的——脚本在独立环境里执行，跑的过程中你的会话依然能响应。这套隔离机制也带来一组必须了解的硬约束。

并发是有上限的：最多 16 个 subagent 同时跑（实际是 $\min(16, \text{CPU 核数} - 2)$ ，核心少的机器会更低），单次运行最多 1000 个 agent。后面这个数字是防止脚本陷入死循环失控烧钱的保险丝。

权限上，Workflow 内部派生的所有 subagent 自动以 `acceptEdits` 模式运行，文件编辑不再逐个弹窗确认，并且继承你当前会话的工具允许列表（tool allowlist）。但有一类情况仍然会打断运行——不在允许列表里的 shell 命令、网络抓取和 MCP 工具，跑到一半还是会向你弹窗确认框。所以官方的建议是：大规模运行之前，先把 agent 们需要用到的命令加进允许列表，免得跑了一半被一个权限弹窗卡住。

还有一点容易被忽略：脚本本身没有直接的文件系统或 shell 访问权限，所有的读写和执行都得通过 subagent 来完成。脚本是纯粹的“调度大脑”，手脚都长在 subagent 身上。

可恢复是 Workflow 相比 subagent、Agent Teams 独有的能力。每次运行都会留一份 journal，改完脚本后用 `resumeFromRunId` 重跑，没改动的 `agent()` 调用直接命中缓存，只有改动及其之后的部分才重新跑。这一点对调试编排逻辑很省事——改一行 prompt 重跑，前面已经跑对的那些 agent 直接命中缓存，不用再花一遍钱和时间。但要注意一条界线：恢复只在同一个会话内有效。运行中途暂停了可以接着跑，已完成的工作不会丢；可一旦你退出了 Claude Code，下次进来这个 Workflow 只能从头再跑一遍。

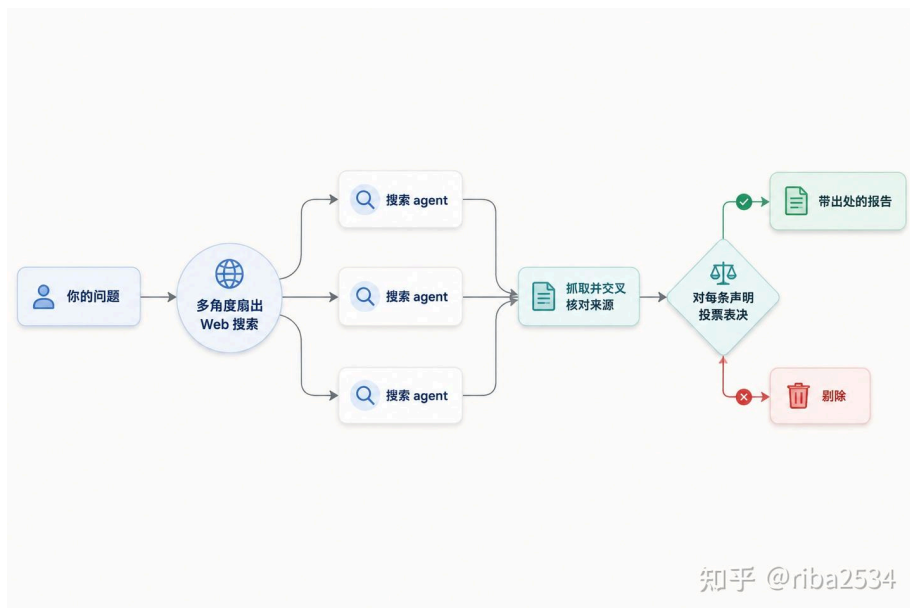
内置的 deep-research 长什么样

光说概念有点抽象，Anthropic 直接内置了一个现成的 Workflow 让你上手体验—— `/deep-research`。

它的用法就是后面跟一个问题：

```
/deep-research <question>
```

跑起来之后，它干的事情分三段：先从多个角度扇出（fan out）一批 Web 搜索，然后抓取并交叉核对这些来源，接着对每一条声明（claim）投票表决，最后产出一份带出处的报告，没通过交叉验证的声明会被直接剔除。



它依赖 WebSearch 工具可用。这个内置 Workflow 的价值在于，它把“对抗幻觉”这件事用编排的方式做了进去——单个 Agent 搜索容易被某个来源带偏，而多路搜索加交叉投票，本质上是在用结构化的流程逼近事实。想感受 Workflow 是什么体验，跑一个 `/deep-research` 是成本最低的入口。

Ultracode：让 Claude 自己决定要不要起 Workflow

如果你不想每次都手动判断“这个任务值不值得一跑 Workflow”，可以把这个决定权交给 Claude——开启 `ultracode`。

```
/effort ultracode
```

这条命令同时做两件事：把推理努力（effort）拉到 `xhigh`，同时允许 Claude 自动判断什么时候该用 Workflow 来处理你的任务。开启之后，一个请求可能被拆成连续好几个 Workflow——比如先跑一个理解代码，再跑一个做修改，最后跑一个验证。这个设置对当前会话里的每个任务都生效，新会话则会重置。

代价也很直白：官方文档说得很清楚，`ultracode` 模式下每个请求消耗的 token 和耗时都明显高于较低的努力档位。想退回日常工作模式，降回去就行：

```
/effort high
```

这种“让模型自己决定调度规模”的设计，跟 Codex 那套 Goals（持久目标）走的是不同的方向，这点最后再细说。

Workflow 是一张 DAG 吗

聊到编排，很多人脑子里第一反应是 DAG（有向无环图）——Airflow、Argo、GitHub Actions 的 `needs:`，都是先把一张静态的依赖图画死，再按图执行。Workflow 是不是也是这样？

答案要看你问的是哪张图：作为程序，它不一定是 DAG；但任何一次执行的轨迹，一定是 DAG。

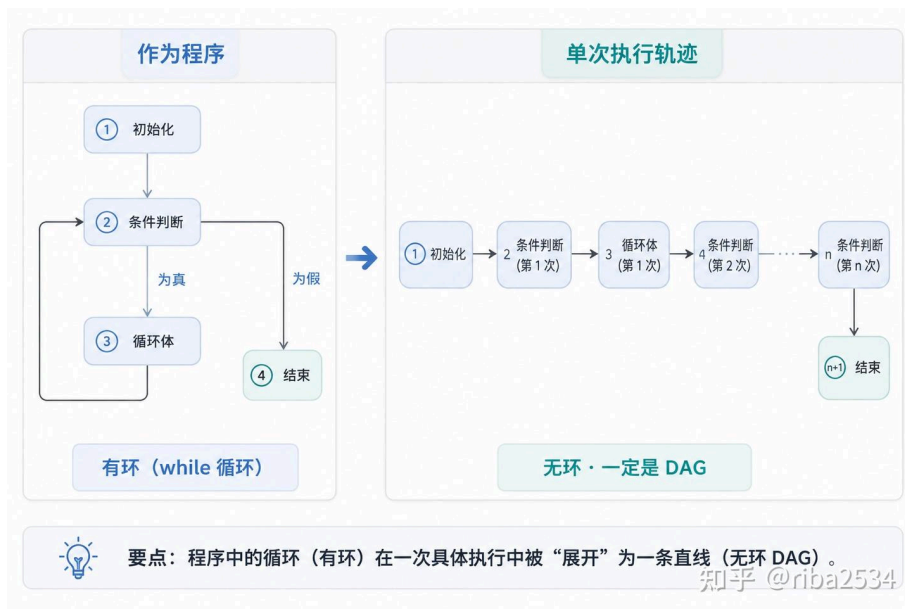
先说程序层面。Claude Code 的 Workflow 是一段图灵完备的命令式 JavaScript，不像传统编排器那样跑之前就把依赖图定死。它能写出 DAG 表达不了的东西，最典型的就循环——比如“一直找 bug，直到连续两轮都没有新增”：

```
let dry = 0
while (dry < 2) {
  const fresh = (await parallel(FINDERS.map(...))).filter(isNew)
  if (!fresh.length) { dry++; continue }
  dry = 0
  confirmed.push(...await verify(fresh))
}
```

// ← 这是一条回边，控制流图里的环

除了循环，它还能写运行时才决定的分支（`if (bugs.length === 0) return`，走哪条路取决于上一步 LLM 的输出），以及动态扇出（下一阶段起几个 agent，取决于上一阶段返回了多少条结果）——图的形状事先并不知道。在“程序结构”这个层面，它有环、有数据决定的分支，比 DAG 严格更强。

但只要你盯住“某一次具体执行”，它又一定是 DAG。原因有两个：数据只往时间前方流，一个值不可能依赖它之后才产生的值；循环会被展开，`while` 第 $N+1$ 轮的那些 agent，和第 N 轮是不同的节点——环在“程序”里，展开成“轨迹”后就被拉直成一条链了。



一句话记住：带 `while` 的程序不是 DAG，但它跑一次产生的执行轨迹，永远是 DAG。这正是它比传统 DAG 编排器更灵活的地方——传统工具的图在跑之前就定死了，而 Workflow 的拓扑是命令式脚本跑出来的，形状运行时才定。

真实案例一：Bun 从 Zig 移植到 Rust

前面所有的概念，都不如 Bun 这个案例有说服力。开头那组数字可以再看一眼：11 天、约 75 万行 Rust、99.8% 的原有测试通过，从第一个 commit 到

Bun 是一个用 Zig 写的 JavaScript 运行时，性能是它的招牌。把这样一个庞大的运行时从 Zig 整体迁移到 Rust，是那种听起来就让人头皮发麻的工程——光是 Rust 借用检查器（borrow checker）对内存所有权的严格要求，就足以让人工迁移举步维艰。Jarred Sumner 用三个串联的 Workflow 把它啃了下来。



阶段一是生命周期映射。第一个 Workflow 专门做一件事：给 Zig 代码库里每一个 struct field 算出它对应的、正确的 Rust lifetime。这一步单独拎出来做，是因为它是后面所有移植工作的地基——Rust 的内存安全建立在生命周期标注之上，这一层没算对，后面写出来的 .rs 文件根本过不了编译。

阶段二是并行文件移植，也是最能体现 Workflow 规模优势的一段。下一个 Workflow 把每个 .zig 文件移植成一个行为等价的 .rs 文件，数百个 agent 同时开工，每个文件还配两个 reviewer 做交叉审查。把这个量级跟 Agent Teams 对比一下就能感受到差距——Agent Teams 同时跑三五个队员就到协调上限了，而 Workflow 在这里是几百个 agent 并行外加双重 review。

阶段三是编译与测试的 fix loop。文件移植完只是半成品，真正的硬仗是让它们能编译、能通过测试。第三个 Workflow 驱动整个 build 和 test 套件，循环修复，直到两者都干净跑过。这正是上一节讲的 while 循环模式的典型场景——靠脚本里的循环逻辑反复迭代，而不靠 Claude 逐轮盯着。

三个阶段各自的特征可以列成一张表：

阶段	输入	产出	Workflow 特征
一·生命周期映射	Zig 代码库	每个 field 的 Rust lifetime 映射	全局分析，为后续铺路
二·并行移植	.zig 文件	行为等价的 .rs 文件	数百 agent 并行 + 双 reviewer
三·编译测试	半成品 Rust 代码	能编译、过测试的代码	循环修复直到收敛

事情到这还没完。移植合并之后，又跑了一个 overnight workflow（过夜工作流）专门处理收尾——扫描代码里不必要的数据拷贝，每发现一处优化就单独开一个 PR 交给别人做最终审查。这种“夜里挂着自己干长尾清理、产出一堆待 review 的 PR”的用法，是 Workflow 很有意思的一个侧面。

需要说清楚的是，官方明确标注了 Bun 的这个 Rust 版本当时还没进入生产环境——这整套流程跑通了、测试过了，但离上线还有距离。Jarred 自己也说，后续会专门写文章讲这件事的更多细节。

真实案例二：用 Workflow 盘点 133 个历史会话

Bun 是个极端案例，普通人未必有机会跑一次跨语言大迁移。我自己拿一个更接地气的任务试了一下：用 Workflow 给自己的 Claude Code 历史会话做一次“使用画像”。

任务是这样的：~/claude 目录下攒了 133 个会话、130MB 的 jsonl 记录，想从里面提炼出使用模式、反复出现的痛点和可以自动化的点。这个活的特点是数据量大、维度多，正好适合 fan-out。

整个任务拆成了“主 Agent 预处理 + Workflow 编排”两段。主 Agent 先做侦察和清洗：jsonl 里塞满了工具调用的噪声，直接喂给 agent 会浪费上下文，于是先写个脚本把 133 个会话压缩成“标题 + 用户真实输入 + 元数据”，得到 601 条真实的人类输入，再切成 10 个批次。然后才是 Workflow 上场：10 个分析 agent 并行各啃一个批次（按统一的 schema 抽取领域分布、卡点、自动化候选），最后 1 个综合 agent 跨批汇总去重，产出一份带优先级的报告。



这次运行的执行体，去掉声明部分后大致是这样：

```
phase('分析')
const batches = Array.from({ length: 10 }, (_, i) =>
  `${DIR}/batch_${String(i + 1).padStart(2, '0')}.md`)

const findings = await parallel(batches.map((path, i) => () =>
  agent(ANALYZE_PROMPT(path), {
    label: `分析:batch_${String(i + 1).padStart(2, '0')}`,
    phase: '分析',
    schema: FINDING_SCHEMA, // 强制结构化输出，不匹配自动重
  })
))

const ok = findings.filter(Boolean)
log(`分析完成: ${ok.length}/${batches.length} 批
```

```
phase('综合')
const corpus = JSON.stringify(ok, null, 1) // 把 10 份发现整个塞进综合 p
const report = await agent(SYNTH_PROMPT, { label: '综合报告', phase: '综合' })
return report // 这个返回值就是唯一回到主上
```

跑下来的账单：11 个 agent、81.8 万 token、254 秒。这中间我还踩了一个坑——第一次跑直接挂了，报 `TypeError: undefined is not an object (evaluating 'batches.map')`，原因是 `args` 没正确传进去、被当成了字符串。修法很能体现 [Workflow“脚本”即文件、可迭代](#) 的价值：我没有重发整段脚本，只是直接 Edit 那个落盘的脚本文件、把路径写死成自包含，再用 `scriptPath` 重跑。

这个案例还顺带回答了一个很多人会问的问题：这事派几个 `subagent` 一样能做，区别在哪？确实能做——区别不在“能不能”，在“编排逻辑放哪、中间结果流到哪”。如果用 Agent 工具派 10 个 `subagent`，10 份结果会作为 `tool result` 全部回到主上下文，然后你得在下一个回合用自己的脑子读完、决定怎么合；编排逻辑是临场判断，每一步协调都烧主上下文的 token。而 Workflow 把编排写成了代码，10 份中间结果不进主上下文，只回最终那份报告，`schema` 自动校验、并发自动管控。

不过得诚实地补一句：就这一次这种“一把梭 `map-reduce`”而言，两者差距其实没那么大——10 路并行、合一次就完了，`subagent` 也够用。Workflow 真正赚到的是随复杂度放大的那部分：当阶段变多、需要循环（一直找到连续 N 轮无新增）、需要多轮对抗式验证、或者要 `fan-out` 到几十个单元时，用 `subagent` 手动协调会非常痛，用脚本写就很自然。

和 N8N、Coze、Dify 的本质区别

看到“用代码编排多步流程 + 步骤里塞 LLM”，很容易冒出一个念头：这不就是 `n8n`、Coze（扣子）、Dify 那套吗？无非是现在让模型自动来编排。

这个直觉抓住了最关键的共性，但“唯一区别是模型自动编排”这句话，得往回拉一拉。

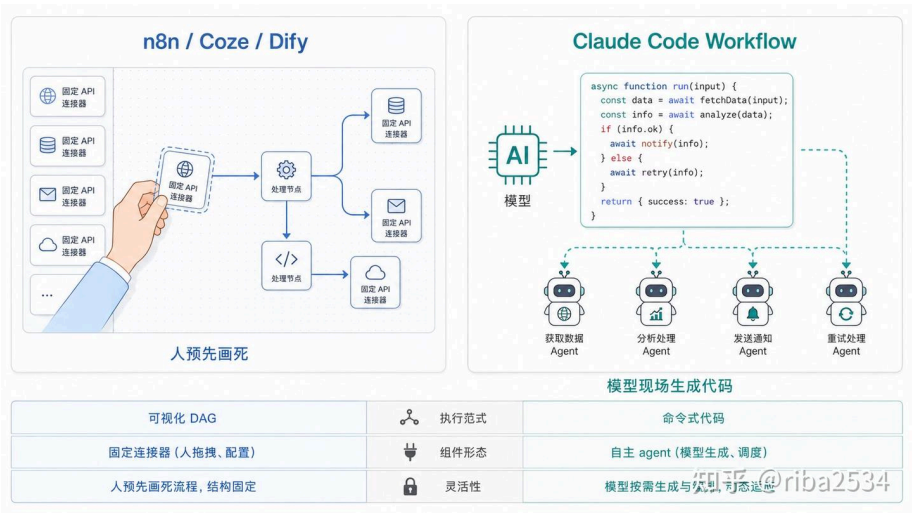
先说共性，而且这个共性比想象的更硬。Anthropic 在《Building Effective Agents》里给过一个权威定义：Workflows 是 LLM 和工具通过“预定义代码路径”（`predefined code paths`）被编排的系统；与之相对的 Agents，才是 LLM 在运行时动态指挥自己流程的系统。按这个定义，Dynamic Workflow 和 `n8n`/Dify/Coze 是同一类——它们的控制流都是确定性的，LLM 不会在运行时决定“下一步走哪条边”，脚本一旦写好，执行它的就是个无脑运行时，LLM 只在节点内部干活。真正不在这个范畴里的，是主对话那个 ReAct 式 agent（那才是模型实时决定下一步）。这一点的判断完全成立。

但“唯一区别”这个说法漏掉了两个更硬的差异。把它们摊开看：

维度	n8n / Dify / Coze	Claude Code Workflow
谁写、何时写	人事先拖一次，之后反复跑（可复用产品）	模型针对单次任务现写一段脚本（也能存成命令复用）
流程的表示	可视化节点图 / 声明式配置，本质是静态 DAG	命令式代码（JS），图灵完备
能表达的形状	大体是静态 DAG，循环分支靠专门节点，拓扑设计时定死	while、数据决定的动态扇出，拓扑运行时才定
“节点”是什么	一个固定操作（HTTP / 转换 / 一次固定的 LLM 调用）	一个完整自主的 sub-agent（能用工具、读文件、跑 bash、多轮推理）
运行环境	独立平台，webhook / 定时触发，面向自动化	嵌在交互式编码会话里，共享 session 的工具、模型、预算

把这张表压成一句话：Workflow ≈ 把 `n8n` 那张图，换成模了两样东西——作者（人 → 模型）和载体（可视化 DAG →

和定制性（不用人预先搭，针对当次任务量身生成），第二样带来表达力的提升（能写循环和动态扇出，这是可视化 DAG 做不到的）。



至于“AI 自动编排”这个最抢眼的差异，还得再精确一点：AI 的介入发生在“写代码”那一刻，不在“跑流程”那一刻。n8n 是人写编排、运行时确定性执行；Workflow 是模型写编排、运行时确定性执行——执行期间模型在睡觉。两者跑流程的方式是一样的，差别只在编排脚本的作者是谁。

值得一提的是，两边其实在互相靠拢：Coze、Dify 都在加 agent 节点（节点本身变自主）和 code 节点（可以写 JS / Python），往“代码 + 自主节点”挪；而 Workflow 的脚本也能存进 .claude/workflows/ 当可复用产品，往“建一次反复用”挪。所以更准确的结论是：你的本质判断成立——都是确定性流程编排、LLM 当步骤；但区别不止“AI 自动编排”一条，还有载体是图灵完备代码而非可视化 DAG，以及每个节点是个自主 agent 而非固定连接器。

官方之前，怎么手搓一个 Workflow

既然 Workflow 本质是“确定性脚本 + 在节点处调 LLM”，那在官方推出之前，自己手搓一个类似的东西，完全可行。核心拼图就一个：claude -p。

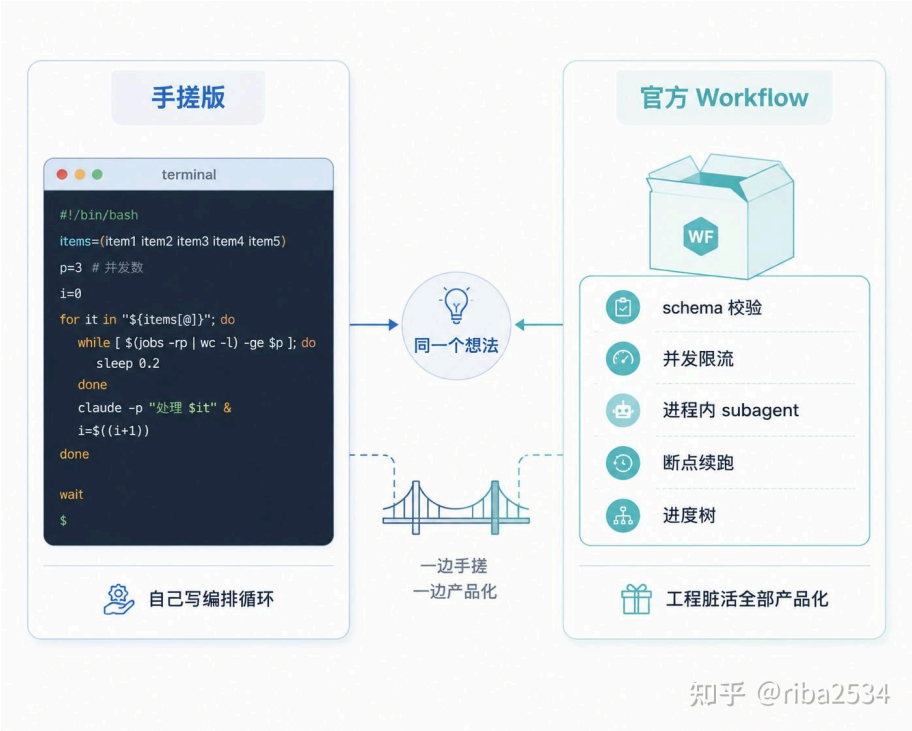
claude -p（即 --print，headless 模式）会非交互地跑完一整个 agent loop——思考、调工具、改文件——跑完即退出。它读 stdin、写 stdout，可以像普通命令行工具一样接进管道。把每一步当成一次 claude -p 调用，外面用 shell 或 Python 写编排循环，就是 DIY 版的 Workflow：

```
# fan-out: 10 个批次并行，每个起一个 claude -p
for f in batch_*.md; do
  claude -p "分析这个批次: $(cat $f)" --output-format json > "out_$.json" &
done
wait

# reduce: 把 10 份结果拼起来，再起一个 claude -p 综合
claude -p "综合这些发现: $(cat out_*.json)" > report.md
```

对照一下就会发现，这跟前面那个 133 会话的 Workflow 是同构的：& 加 wait 就是 parallel() 的屏障，\$(cat ...) 拼字符串就是 prompt 里的变量插值。社区里这类实践不少，futuresearch.ai 那篇就用 claude -p 加文件系统轮询搭了一套 18 路并行的扫描流水线——子 agent 把结果写盘（成功写 .json、失败写 .error），编排器只轮询文件名而不把输出收进上下文，把复杂度从 $O(n \times \text{输出大小})$ 压到 $O(n \times \text{文件名})$ 。

那官方 Workflow 比手搓版多了什么？答案是：模型没变，省掉的全是工程脏活



手搓版你得自己写	官方 Workflow 内置
--output-format json 之后自己校验、不对就重试	schema: 自动校验 + 失败自动重试
用 xargs -P 或信号量控并发，别把机器打爆	自动按 min(16, 核数-2) 限流
每个 claude -p 是独立进程 + 独立 session，重新加载配置、MCP、登录态，很重	agent 是进程内轻量 subagent，共享当前 session 的模型和工具
挂了从头再跑	按 runId 缓存，改脚本只重跑变化的部分
进度、token 统计自己埋点	/workflows 看进度树 + token 账一处汇总

一句话：Workflow 是把这套手搓 harness 产品化了，省的是工程脏活，不是改变模型。理解了这一层，你对它的能力边界也就有了底——它不是什么魔法，就是一个把" claude -p + 编排循环"打磨得足够顺手的运行时。

什么场景该用 Workflow

不是每个任务都需要起一个 Workflow。它本质上是用大量并行 agent 换效率，而并行 agent 是实打实烧 token 的。那什么时候这笔账划算？

官方文档给出的适用场景集中在四类。

- 一是代码库范围的批量排查，比如全仓库 bug 扫描、性能剖析引导的优化审计、安全审计。这类任务的共同点是“搜索加独立验证”——Claude 并行搜遍整个服务，再对每个发现单独验证，确保报告里浮现的都是真问题。授权检查、输入校验、危险模式的全库加固也是同一个形状。
- 二是大规模迁移与现代化，框架替换、API 弃用迁移、跨语言移植都算，Bun 是其中最极致的例子。
- 三是需要反复推敲的关键决策。当错误答案的代价很高时，让 Claude 从多个独立角度各做一遍，再派对抗性的 agent 试图推翻这些结果，迭代到答案收敛为止——这种对抗式验证能逼近单次跑达不到的质量。
- 四是长尾清理，像 Bun 那个 overnight workflow，挂着自动扫描问题、逐个开 PR。

反过来，这几类任务用 Workflow 就是杀鸡用牛刀：一两步
繁拍板的探索性工作、以及碰安全和支付这类高风险代码的

把 Claude Code 现有的几种协作原语放一起，选型逻辑大致是这样：

你的任务	合适的原语
一两步的小改动、明确的单点任务	单个 session
派出去搜资料、读代码，拿结果就行	Subagent
几个维度并行、队员之间需要讨论	Agent Teams
几十上百个并行、要循环和交叉验证、想把编排存下来复用	Workflow

一句话区分：需要“跑腿”用 subagent，需要“开会讨论”用 Agent Teams，需要“流水线作业”用 Workflow。

可用范围与怎么开

Dynamic Workflows 目前是研究预览状态，对版本和计划都有要求。

版本上需要 Claude Code v2.1.154 或更高。平台覆盖得相当全：CLI、Desktop、VS Code 扩展、Claude API，以及 Amazon Bedrock、Google Cloud Vertex AI、Microsoft Foundry 三大云。

计划上的差异要留意：Max、Team 以及通过 API 使用的场景，默认就开着；Pro 计划默认关闭，需要去 /config 里的 Dynamic workflows 一行手动打开；Enterprise 计划发布时也默认关闭，得由管理员在设置里开启。第一次触发 Workflow 时，Claude Code 会把即将运行的内容摆给你看并要求确认，不会闷头就跑。

如果你想彻底关掉它，有几个层面的开关：

```
// ~/.claude/settings.json
{
  "disableWorkflows": true
}
```

或者用环境变量，在启动时生效：

```
export CLAUDE_CODE_DISABLE_WORKFLOWS=1
```

个人也可以直接在 /config 里切换，组织层面则能通过 managed settings 或管理后台统一禁用。

Token 成本：必须算的一笔账

这部分官方说得格外坦诚：单次 Workflow 的 token 消耗，明显高于一次普通的 Claude Code 对话。道理不复杂，几十上百个 subagent 同时跑，每个都在烧 token，再叠加交叉验证、对抗式 review 这些“额外冗余”的设计，账单自然往上走。前面那个 133 会话的案例，11 个 agent 就吃掉了 81.8 万 token——这还只是 10 路一把梭的轻量编排。这些消耗都计入你在计划的用量和速率限制。

官方给的实用建议有几条可以记下来。一是从一个范围明确的小任务开始，先摸清楚它在你工作里大概花多少，再决定要不要放手让它跑大活。二是大规模运行之前，先用 /model 确认自己在合适的模型上，并且可以要求 Claude 在那些不需要最强能力的阶段改用更小的模型——不是每一步都得用最贵的脑子。三是前面提过的，提前把命令加进允许列表，避免中途被权限弹窗打断一个跑了几小时的任务。

好在 Workflow 随时可以叫停，已经完成的工作不会白费。

已知限制

研究预览阶段，几个边界先摆在这，省得踩到了再回来查：

- 运行中途不接受人工输入：除了权限确认弹窗，Workflow 跑起来就不停下来等你拍板。需要分段签核的流程，得拆成多个独立的 Workflow。
- 脚本本身没有文件和 shell 访问权限：读写和执行全靠 subagent，脚本只负责调度。
- 并发与总量都封顶：最多 16 个并发 subagent，单次运行 1000 个 agent 上限。
- 跨会话不可恢复：退出 Claude Code 后，下次进来 Workflow 从头再跑。
- 自定义 Workflow 怎么传参：内置的 /deep-research 能接收一个问题参数，但你自己存的 Workflow 怎么传参，官方文档这块还语焉不详。
- 整个功能仍在研究预览，行为和约束都可能随版本调整。

它在 Claude Code 体系里的位置

到这里，可以把 Claude Code 的几种扩展原语放在一张表里通盘看一遍：

原语	你提供什么	谁在编排	典型规模
Slash Commands	一段固定的提示词模板	你	单次
Subagents	一个工作者的定义	Claude 逐轮	每轮几个
Skills	一套可复用的指令	Claude 按指令	每轮几个
Agent Teams	团队结构和分工	Team Lead	几个队员
Workflows	一个任务描述	脚本	数十到数百

这张表从上到下，编排权一步步从“你”转移到“Claude”，再转移到“代码”。Workflow 站在最右端——你只给一句任务描述，编排的活儿交给 Claude 写的脚本，规模冲到数百个 agent。

这里就要回到开头那个伏笔：为什么 Dynamic Workflows 和 Opus 4.8 同一天发布。当你让几百个 agent 并行干活、还要它们互相 review 彼此的结论时，每个节点的可靠性就被放大了——节点是概率性的 LLM，同一个问题换个角度问都可能给出不一致的答案，而多步流程里的不确定性会层层累积。Opus 4.8 这一代专门强化了这块：官方说它“让代码缺陷悄悄溜过、不被发现的概率比前一代低了约四倍”，更倾向于主动标记自己拿不准的地方。这种诚实性的提升，正是“数百 agent 交叉验证”这套打法能成立的前提——交叉验证要管用，前提是 reviewer 真的会指出问题，而不是一味点头。强模型不是 Workflow 的可选项，是它的承重墙。

最后把 Workflow 放进更大的图景里看。它把“编排逻辑代码化”作为一个明确的产品选择，这件事本身值得玩味：编排从模型每次现想现做的临时行为，变成了一段可以读、可以审、可以存、可以反复跑的代码资产。把它和 Codex 的 Goals 摆在一起看，会发现一个有趣的分野。两者都想解决“大任务怎么持续推进”，但路径相反：Codex Goals 押注的是目标持久化——把目标钉在那，让模型自己想办法一步步逼近；Claude Code Workflow 走的则是编排代码化这条路——把过程写成脚本，靠脚本保证流程不跑偏。一个管“往哪走”，一个管“怎么走”，是两套不同的工程哲学。哪条路最后跑得更远现在下结论还太早，但两家头部产品在“承载超大规模工作”这个问题上同时发力，本身就说明这是 AI Coding 下一阶段的主战场。

最后，说点我自己的判断。我觉得 dynamic workflow 这套东西相当强大，大概率代表了未来的方向。它能站住，靠的是两块拼图严丝合缝：一块是把“编排”从模型的临场发挥固化成可控的代码，另一块是一个足够诚实、足够强的前沿模型，撑得起几百个 agent 互相交叉验证——这也正是它要跟 Opus 4.8 同天登场的原因。前沿模型是地基，编排代码化则是让“几百个 agent 协作还不跑偏”成为可能的关键，两者缺一不可。

也正因为它对前沿模型的能力依赖得这么深，我的判断是：一年之内，这套“模型现写编排脚本、再调度一支 agent 舰队”的打法，会从某一家的研究预览，长成几乎所有 coding agent 的标配。

顺带一提，这篇文章背后的调研，本身就是一个 Workflow 跑出来的：十五个 agent 并行分头干活——精读一手对话记录、交叉比对业界资料——27 万 token、169 秒，最后汇成一份素材交回来。用 Workflow 把 Workflow 讲清楚，大概是对它

参考资料

- [Dynamic workflows 官方文档](#)
- [Introducing dynamic workflows in Claude Code \(官方发布博客\)](#)
- [Claude Opus 4.8 发布说明](#)
- [Run agents in parallel \(官方对比 subagent / agent teams / workflow\)](#)
- [Building Effective Agents \(Anthropic 对 workflow 与 agent 的定义\)](#)
- [Headless / claude -p 官方文档](#)
- [Bun "Rewrite Bun in Rust" PR \(oven-sh/bun #30412\)](#)

发布于 2026-05-29 05:00 · 日本

[claude](#)